An Efficient Finite Element Method for Submicron IC Capacitance Extraction*

N.P. van der Meijs, A.J. van Genderen

Delft University of Technology
Department of Electrical Engineering
Mekelweg 4, 2628 CD Delft, The Netherlands

Abstract

We present an accurate and efficient method for extraction of parasitic capacitances in submicron integrated circuits. The method uses a 3-D finite element model in which the conductor charges are approximated by a piece-wise linear function on a web of edges located on the surface of the conductors. This yields a system of Green's function integral equations that is solved by a novel approximate matrix inversion technique that only utilizes the entries corresponding to pairs of finite elements that are physically close to each other. With N representing the size of the layout, this results in time and space complexities of $O(N)$ and $O(\sqrt{N})$ respectively. The method has been implemented in an efficient layout to circuit extractor and experimental results are presented.

Introduction

With the decrease of feature sizes and the increase of chip dimensions in IC technology, the influence of interconnect capacitances on circuit performance is becoming more prominent. Therefore, the need for accurate extraction of interconnect capacitances increases. At the same time, however, this problem becomes more difficult because lateral dimensions decrease more rapidly than vertical dimensions. Thus, conventional area/perimeter based estimations of capacitances become inaccurate. Instead, rigorous mathematical techniques are required to obtain sufficiently accurate estimates of the interconnection capacitances.

Possible techniques for accurate capacitance extraction include the finite difference method [1, 2, 3, 4], the finite element method in which the finite elements model the electrical field [5], and the boundary or Green's function finite element method [6, 7, 8, 9], in which the finite elements model the conductor charges. In this paper, we describe a variant of the Green's function technique which, combined with an efficient method for approximate matrix inversion, enables 3-dimensional capacitance extraction in $O(N)$ time and $O(\sqrt{N})$ space, where N represents the size of the layout. We also describe how these techniques have been implemented in a layout to circuit extractor that actually achieves these bounds.

Briefly, the capacitance extraction method presented in this paper consists of:

1. Discretizing the charge that is present on conductors as a linear and continuous distribution on a web of edges located on the surface of the conductors. The basic elements in this discretization are so-called *spiders:* sets of edges that are incident on a given node [10, 11].
2. Using a green's function technique, setting up a set of equations relating spider potentials to spider charges. This set of equations can be written as:

$$\phi = G\,\sigma \tag{1}$$

where $\phi = [\phi_1\ \phi_2\ \cdots\ \phi_N]^T$ and $\sigma = [\sigma_1\ \sigma_2\ \cdots\ \sigma_N]^T$ collect the spider potentials and the spider charges respectively, and G_{ij} is the potential induced at node i by the charge at spider j. In finite element modeling, G is sometimes called the elastance matrix.
3. Computing the conductor capacitances as follows: Let A be an incidence matrix relating spiders to conductors, i.e. A_{ij} is 1 if spider i lays on conductor j, and 0 otherwise. Let also $\Phi = [\Phi_1\ \Phi_2\ \cdots\ \Phi_M]^T$ be the vector of conductor potentials and $Q = [Q_1\ Q_2\ \cdots\ Q_M]^T$ the vector of charges on the conductors. Then:

$$Q = A^T\sigma = A^T G^{-1}\phi = A^T G^{-1} A\,\Phi \tag{2}$$

Hence,

$$C_s = A^T G^{-1} A \tag{3}$$

where the matrix C_s is the so-called short-circuit capacitance matrix [7], relating the conductor charges and potentials. The network capacitances are derived from the short circuit capacitances as follows [7]:

$$C_{ij} = -C_{s\,ij} \text{ for } i \neq j, \qquad C_{ii} = \sum_{j=1}^{M} C_{s\,ij} \tag{4}$$

C_s is a full matrix, that is, it specifies a capacitance between every pair of conductors. Clearly, this corresponds to a circuit that is too complex for sensible verification. Moreover, computational complexity would restrict the applicability of the method to small test cases without any practical significance.

As a solution, our method exploits a novel technique to compute only a sparse approximation of G^{-1}, thereby in effect ignoring small capacitances between conductors that are physically "far" from each other. The time complexity of the approximate matrix inversion technique is optimal: proportional to the square of the number of pairs of nearby spiders. Consequently, since the average density of spiders in actual layouts is constant, the running time is proportional to the size of the layout.

The Schur Algorithm

In equation (3), G^{-1} is the capacitance matrix of the system of elementary conductors represented by the elementary spiders. Although in principle G^{-1} is a full matrix, we are interested in a sparse approximation of it that contains zeros at positions (i,j) when the physical distance between spider i and spider j is larger than some constant related to the desired accuracy. An

* This research was supported in part by the commission of the EC under ESPRIT contract 991 and by the Dutch FOM under IOP-IC contract 45.009.

algorithm has been discovered [12], the generalized Schur algorithm, that computes such an approximation with a complexity that is determined by the number of sparse entries. It utilizes only entries in those positions (i,j) of the original matrix G for which non-zero entries in the approximate inverse G^{-1} are desired.

The algorithm produces an approximate positive definite inverse G_{ME}^{-1} for a positive definite matrix G that is specified on a staircase band S, or that can be permuted to be specified on such a band. This approximate inverse is called G_{ME}^{-1} since it is actually the inverse of the so-called maximum entropy extension of G [13], which is the unique positive definite matrix that coincides with G on S and of which the inverse is zero on the complement of S. The algorithm requires $O(Nb^2)$ operations, where b is the average width of the staircase band.

The Hierarchical Schur Algorithm

For circuit extraction purposes, the support S of G depends on the numbering scheme of the spider elements. It is, in general, not possible to number the spiders in such a way that the set of all pairs of indices of nearby spiders is in a staircase set, and this set contains no index pairs corresponding to spiders that are far apart. Therefore, the Schur algorithm cannot be used directly.

However, in [14] it is shown how to extend the generalized Schur algorithm to obtain a "hierarchical banded inverse" of G, where G is specified on a multiple band. The hierarchical banded inverse of G has zeros on all positions corresponding to unspecified entries in G, and is the inverse of a matrix that closely matches G on its specified entries. As we will show, this modified Schur algorithm can be used for extraction.

In order to describe this algorithm, we introduce the following notation first. Block matrices and entries of block matrices are denoted by bold uppercase letters. For a block matrix $\mathbf{G}$, the notation $\mathbf{G}(i,j)$ denotes the principal block matrix that lies in the rows and columns of $\mathbf{G}$ indexed by $i,\ldots,j$. The symbol $\nabla(\mathbf{G},(i,j))$ denotes the block matrix that is such that $\nabla(\mathbf{G},(i,j))(i,j) = \mathbf{G}$, and is zero on the other parts of $\nabla(\mathbf{G},(i,j))$. The size of $\nabla(\mathbf{G},(i,j))$ will be defined by its context. In other words, $\mathbf{G}(.,.)$ takes a block out of a matrix and $\nabla(\mathbf{G},(.,.))$ embeds a matrix into a larger matrix such that it is surrounded by zeros.

Now, let G be a positive definite matrix that is specified on a multiple band. Let it be partitioned as $\mathbf{G} = [\mathbf{G}_{ij}]$, $i,j = 1,\ldots,N$, where the blocks $\mathbf{G}_{ij}$ are of size $n_i \times n_j$, such that, for some band with support $\{(i,j) \mid |i-j| \le M\}$, the partially specified principal submatrices $\mathbf{G}(j,j+M)$, $j = 1,\ldots,N-M$, can be permuted to positive definite matrices that are specified on a staircase band, and such that the blocks outside the band are unspecified.

In [14] the hierarchical banded-inverse approximation of G, denoted by G_{HBI}, is defined as

$$\mathbf{G}_{HBI} = \sum_{j=1}^{N-M} \nabla(\mathbf{G}(j,j+M)_{ME}^{-1},(j,j+M)) \qquad (5a)$$

$$- \sum_{j=1}^{N-M-1} \nabla(\mathbf{G}(j+1,j+M)_{ME}^{-1},(j+1,j+M)) \qquad (5b)$$

It is shown there that G_{HBI} provides for a good approximation of G^{-1} with the desired sparsity pattern. An example of how equation 5 would produce the hierarchical banded inverse of a block matrix with $N = 3$ and $M = 1$ is given in figure 1.

Although the hierarchical banded inverse of G can not be guaranteed to be positive definite, it will fail to be so only if the completely specified matrix G is close to singular. In practice, the algorithm has been observed to perform well; results are presented in a later section.

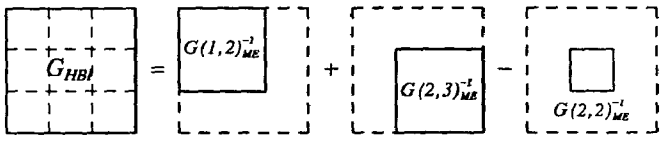

Figure 1. An example of equation 5 for N = 3 and M = 1.

Three Dimensional Capacitance Extraction

We are now ready to present the extraction algorithm. This algorithm is implemented in our layout to circuit extractor called SPACE*. SPACE is a scanline based extractor, and we will see that this capacitance extraction method nicely and elegantly fits into that paradigm. With w the distance over which coupling capacitances are considered significant, SPACE defines a window of width $2w$ and height w that is swept over the layout. Within the window, spider elements are assumed to be coupled with all other spiders in the window, the Green's function is computed for all pairs of coupled spiders, and the resulting elastance matrix is inverted on the fly. A more precise description of the algorithm is as follows:

1. **[Scan Layout]** A strip of width $2w$ is moved from left to right over the layout in steps of size w, as a result of which neighboring strips overlap half of their width. For each strip of width $2w$ and for each overlap of strips of width w, do steps 2-6. For strips of width $2w$ these steps correspond to one term in the summation in equation (5a) and for overlaps of strips of width w, these steps correspond to one term in the summation in equation (5b).

2. **[Generate Mesh]** A mesh is created for the layout in the current strip. The mesh is made fine enough to accurately model the charge distribution on the conductor surfaces. To resolve the incidence between spiders and the particular conductors on which they are located, each spider contains a pointer to a node in the circuit datastructure corresponding to the conductor on which the spider is located.

The strip is vertically partitioned into a number of regions of size *yResolution*, which can be of the order $w/10$. Within a region, spiders are collected into a bucket that is associated with that region. The buckets are organized as an array of size *numberOfBuckets*.

3. **[Compute Elastance Matrix]** Now, define the bucket distance of two spiders as the difference of their index into the bucket array, and assume that all spiders that lay within a bucket distance of $\lceil w/yResolution \rceil$ of each other influence each other. This one-dimensional distance measure between spiders will produce an elastance matrix with a staircase band structure that is efficiently invertable with the Schur algorithm in step 4. It neglects the x- and z-components of the real (euclidian) distance between spiders. This is acceptable since the spiders must be close in the x-direction since they are all located in the same strip, and they must be close in the z-direction due to the geometry of the conductors. As an example, figure 2.b shows the staircase band structure of the elastance matrix for the spiders in the bucket array of figure 2.a, where spiders within a bucket distance ≤ 2 of each other are assumed to influence each other.

---

* Actually, 3-dimensional capacitance extraction is only one mode of operation of SPACE, the program also includes area/perimeter based capacitance extraction and finite element based resistance computation methods [15]. The output is a network containing transistors, capacitances and resistances, and can directly be simulated with e.g. SPICE.

A pseudocode for the algorithm which sets up the elastance matrix is shown in figure 3. This algorithm enumerates all pairs of nearby spiders, and calls a function f that actually evaluates the Green's function integral between 2 spiders, and stores the result in position *row*, *col* of the elastance matrix.

Internally, the elastance matrix is stored diagonal-wise. Each row contains the influences between the spider in the first column and those within a bucket distance of $\lceil w/yResolution \rceil$ of it. This is depicted in figure 2.c for the elastance matrix of 2.b.

4. **[Invert Elastance Matrix]** The elastance matrix is inverted using the Schur algorithm. The result is a short-circuit capacitance matrix C_s, with non-zero entries only on the positions for which the Green's function has been specified.

5. **[Update Circuit]** Using the same spider pair enumeration algorithm as in step 3, the capacitances from the C_s matrix are added to the circuit. For this purpose, the function f now extracts the result from position *row*, *col* from the C_s matrix. Its pseudocode is shown in figure 4. In effect, this algorithm implements equations 3 and 4.

There is no explicit pre- and post-multiplication with an incidence matrix. Instead, all capacitances are added to the circuit and the circuit module accumulates the sums of capacitance values between distinct nodes and discards capacitances between connected nodes. When nodes become connected later, in case a new connection between two pieces of interconnect has been found, coupling capacitances between these nodes are also discarded.

In case the current strip is of width w, i.e. it is an overlap of two strips, the capacitances are *subtracted* from the circuit by having the algorithm in figure 4 first negating the C_s values. See also the last remark made under step 1.

6. **[Clean Up]** As the strip moves, the spiders to the left of the strip are destroyed. Thus, the finite element mesh is only present in a sliding strip of width $2w$. □

The time and space complexities of this algorithm are given in table 1. For contrast, the table also shows the cost of a method based on complete inversion of the elastance matrix. In this table, W denotes the width and H the height of the layout. With $N = O(W \times H)$ representing the size of the layout, the Hierarchical Schur algorithm requires $O(N)$ time and $O(\sqrt{N})$ space.

Table 1. Time and space complexities

algorithm	cpu time	memory
complete inversion	$H^3 * W^3$	$H^2 * W^2$
Hierarchical Schur	$H * W$	H

Accuracy and Performance

We will now present some experimental results showing the accuracy and efficiency achievable with the new algorithms as embedded in the SPACE circuit extractor. First consider the 5 parallel conductors shown in figure 5. The mesh that was created for this geometry consisted of 220 spiders, located 1 μ apart on top and bottom of the conductors.

Table 2 shows the results for windows of varying size. One can see that reasonably accurate results for C_{11} and C_{12} are already obtained with a window size of 1.5μ*3μ and very low cpu-time and memory size. The finite element numbering scheme and the support of the elastance matrix for this case are shown in figure 6 (a) and (b) respectively.

The short-circuit capacitance C_s is the sum of ground and coupling capacitances for a conductor (the total capacitive load it

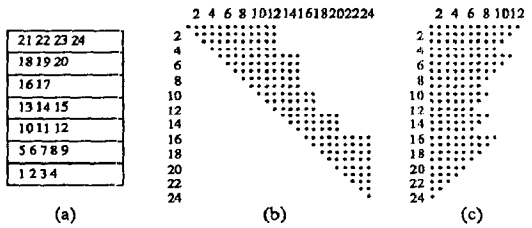

Figure 2. A bucket array and the structure of the elastance matrix

```
row := 0, col := 0
for k := 0 · · · numberOfBuckets−1
    s1 := bucketArray[k]
    while s1 ≠ null
        s2 := s1
        while s2 ≠ null
            f(row, col, s1, s2)
            s2 := s2.next, col := col + 1
        for l := k + 1 · · · Min (k + ⌈w/yResolution⌉, numberOfBuckets−1)
            s2 := bucketArray[l]
            while s2 ≠ null
                f(row, col, s1, s2)
                s2 := s2.next, col := col + 1
        s1 := s1.next, row := row + 1, col := 0
```

Figure 3. The algorithm to enumerate all pairs of nearby spiders

```
updateCircuit (row, col, s1, s2)
if s1 = s2                          # diagonal entry in matrix
    addCapToNetwork (s1.node, gndNode, Cs[row][col])
else if s1.node = s2.node           # spiders are on same conductor
    addCapToNetwork (s1.node, gndNode, 2 * Cs[row][col])
else                                # spiders are on different conductors
    addCapToNetwork (s1.node, gndNode, Cs[row][col])
    addCapToNetwork (s2.node, gndNode, Cs[row][col])
    addCapToNetwork (s1.node, s2.node, -1 * Cs[row][col])
```

Figure 4. The algorithm to convert C_s capacitances to network capacitances

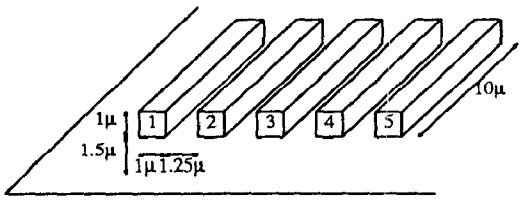

Figure 5. Conductor geometry for the measurements shown in figure 5.

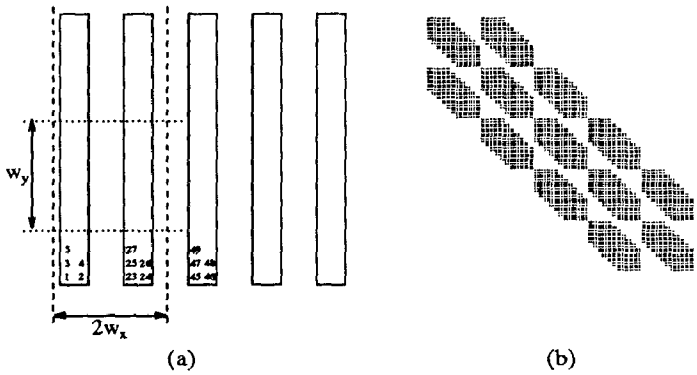

Figure 6. Finite element numbering (a) and support of the matrix for the Hierarchical Schur algorithm (b).

forms under the assumption that the other conductors have a low impedance) and largely determines the timing of the circuit. Note that this capacitance is already accurate for small window sizes. In fact, one can see that if the window size decreases, the "nearby" capacitances slightly increase and thereby compensate

Table 2. Capacitance values and cpu times on an HP 9000-840 for the geometry of figure 5.

window	capacitances (10^{-18} F)						cpu time	memory
(μ)	C_{11}	C_{12}	C_{13}	C_{14}	C_{15}	$C_{\infty 1}$	(min:sec)	(kbyte)
5*10	1128	557.0	37.59	14.93	9.13	1746	2:06.0	2441
4*8	1134	556.9	37.70	17.21		1746	1:30.0	1196
3*6	1148	556.7	42.79			1748	46.1	526
2*4	1184	568.4				1752	14.6	169
1.5*3	1226	530.9				1757	5.9	77
1*2	1578					1578	1.9	27

Table 3. Computation times and memory usage on an HP 9000-840 computer.

			Complete inversion with LU			Hierachical Schur, 2μ*4μ window		
nr of conductors	length (μ)	nr of spiders	nr of Greens	cpu time (min:sec)	memory (kbyte)	nr of Greens	cpu time (min:sec)	memory (kbyte)
5	10	220	24310	2:11	1152	12250	14	169
5	20	420	88410	16:08	4200	26150	33	322
10	20	840	353220	176:14	16800	55848	1:06	322
14	29	1680	*unable to compute*			122340	2:26	461
20	41	3360				253872	5:14	645

for not computing the "far" capacitances.

Table 3 shows computation times and memory usage for an increasing number of conductors of increasing length. Except for their number and length, the geometry of the conductors and location of the spiders is the same as in figure 5. Clearly, the measurements agree with the time and space complexities as expressed in table 1. One can see that the cpu-time for the Hierarchical Schur Algorithm increases linearly with the size of the layout when the size of the window is kept constant.

Although these results may be termed spectacular, they do not even tell the whole story: when the solution of a problem requires large storage of data, computer time is lost because of exhaustive swapping. For example, the third LU test using 17 Mbyte of memory took more than 24 hours of realtime to complete on an otherwise lightly loaded system. Since SPACE is very parsimonious with storage, swapping is mostly avoided.

Finally, table 4 shows results for an extraction of a static ram cell in a 1 micron cmos technology, using an 3μ*3μ window. The mesh that was created for this circuit is shown in figure 7. The results show that the new extraction algorithms as embedded in SPACE indeed enable accurate and rigorous extraction of real submicron layouts, even on workstations and minicomputers.

Table 4. Cmos static ram cell extraction statistics on an HP 9000-835

total cpu time (min:sec)	10:41
time for matrix inversion	5:25
time for green evaluation	4:34
total memory (Mbyte)	6.9
# finite element spiders	1550
# green evaluations	411286

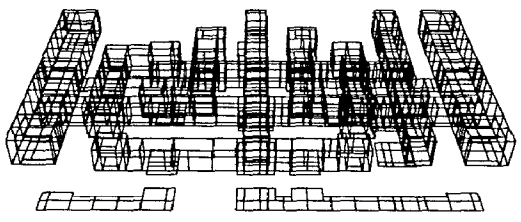

Figure 7. Finite element mesh for a cmos static ram cell as generated by SPACE.

References

1. R.L.M. Dang and N. Shigyo, "Coupling capacitances for two-dimensional wires," *IEEE Electron Device Letters* **EDL-2**(8) pp. 196-197 (Aug. 1981).
2. W.H. Dierking and J.D. Bastian, "VLSI parasitic capacitance determination by flux tubes," *IEEE Circuits and Systems Magazine*, pp. 11-18 (Mar. 1982).
3. A. Seidl, M. Svoboda, J. Oberndorfer, and W. Rosner, "CAPCAL - A 3-D Capacitance Solver for Support of CAD Systems," *IEEE Trans. on CAD* **CAD-7**(5) pp. 549-556 (May 1988).
4. R. Guerrieri and A.L. Sangiovanni-Vincentelli, "Three Dimensional Capacitance Evaluation on a Connection Machine," *Proc. IEEE ICCAD-87*, Santa Clara, pp. 446-449 (Nov. 9-11 1987).
5. P.E. Cottrell and E.M. Buturla, "VLSI Wiring Capacitance," *IBM J. Res. Develop.* **29**(3) pp. 277-288 (May 1985).
6. A.E. Ruehli, "Survey of computer-aided electrical analysis of integrated circuit interconnections," *IBM J. Res. Develop.* **23**(6) pp. 626-639 (Nov. 1979).
7. A.E. Ruehli and P.A. Brennan, "Capacitance models for integrated circuit metallization wires," *IEEE Journal of Solid-State Circuits* **SC-10**(6) pp. 530-536 (Dec. 1975).
8. Z.Q. Ning, P.M. Dewilde, and F.L. Neerhoff, "Capacitance Coefficients for VLSI Multilevel Metallization Lines," *IEEE Trans. on Electron Devices* **ED-34**(3) pp. 644-649 (March 1987).
9. S.P. McCormick, "EXCL: A circuit extractor for IC designs," *Proc. 21st Design Automation Conference*, pp. 616-623 (1984).
10. Z.Q. Ning and P.M. Dewilde, "An Efficient Modelling Technique for Computing the Parasitic Capacitances in VLSI Circuits," *Proc. ISCAS-88*, pp. 1131-1134 (June 1988).
11. Z.Q. Ning and P. Dewilde, "SPIDER: Capacitance Modelling for VLSI Interconnections," *IEEE Trans. on Computer-Aided Design, to be published*, (12)(December 1988).
12. P. Dewilde and E. Deprettere, "Approximate Inversion of Positive Matrices with Applications to Modelling," *Nato ASI Series, Modelling, Robustness and Sensitivity Reduction in Control Systems*, (1987).
13. H. Dym and I. Gohberg, "Extensions of band matrices with band inverses," *Linear algebra and its applications*, (36)(1981).
14. H. Nelis, E. Deprettere, and P. Dewilde, "Approximate Inversion of Positive Definite Matrices, specified on a Multiple Band," *Proc. SPIE 88*, San Diego,(Aug. 1988).
15. A.J. van Genderen and N.P. van der Meijs, "Extracting Simple but Accurate RC Models for VLSI Interconnect," *Proc. ISCAS-88*, Helsinki, Finland, pp. 2351-2354 (June 7-9, 1988).